



★Y OURWARDROBE

★FABRIZIO ★FUSARRI

★SIMONE ★FRIJIO

★ANDREA ★TURRA

★SAMUELE ★VILLA

Introduzione.....	3
Strumenti di lavoro.....	3
Funzionalità.....	4
Autenticazione.....	4
Home Fragment.....	5
Garment Fragments.....	6
Clothes Fragment.....	6
Add Garment Fragment.....	7
Bulk Import Fragment.....	8
Garment Fragment.....	9
Outfit Fragments.....	10
Outfit Menu Fragment.....	10
Create Outfit Fragment.....	11
Single Outfit Fragment.....	12
Personal Stylist Fragment.....	13
Profile Fragment.....	13
Architettura.....	15
Struttura dei package.....	16
model.....	16
adapter.....	16
data.....	16
repository.....	16
remote.....	16
dto.....	17
ui.....	17
di.....	17
Design.....	18
Layout.....	19
Testing.....	20
Sviluppi Futuri.....	21

Introduzione

yourWardrobe è un'applicazione progettata per un ampio spettro di utenza, in particolare per gli amanti della moda e per i “maniaci del controllo” dei propri vestiti. L'applicazione offre un'esperienza intuitiva all'utente, permettendo di salvare e scorrere tra i propri **vestiti**, opportunamente divisi in categorie, comporre e modificare **outfit** e, infine, farseli generare da un personal stylist, in grado di creare outfit adeguati ad ogni contesto. Inoltre, l'applicazione è in grado di consigliare all'utente outfit in base al meteo e di programmare un outfit in vista di futuri appuntamenti.

Strumenti di lavoro

L'IDE utilizzato è stato **Android Studio**, attraverso il linguaggio di programmazione Java.

La piattaforma centrale per lo sviluppo iterativo del codice è stata **Github**, comodamente integrato in Android Studio. Ha avuto un ruolo imprescindibile nel versioning e in una corretta separazione delle responsabilità fra i membri del gruppo. I branch sono stati così organizzati:

- **master** è stato utilizzato per contenere la versione finale dell'applicazione
- **develop** è stato utilizzato come ambiente di sviluppo condiviso, contenitore delle versioni stabili intermedie
- ogni funzionalità aggiunta strada facendo è stata programmata in un branch dedicato, chiamato **feature/home_feature**. Una volta considerate stabili, sono state *merged* nel branch principale develop.

La nostra scelta per la persistenza delle informazioni e per l'autenticazione dell'utente è stata la piattaforma cloud di **Google Firebase**, sfruttando **Firestore Database** per la memorizzazione di informazioni riguardanti l'utente, i suoi vestiti e i suoi outfit; **Firebase Storage** per salvare le foto dei vestiti e **Authentication** per gestire l'autenticazione degli utenti (implementata con **email + password** e **accedi con Google**).

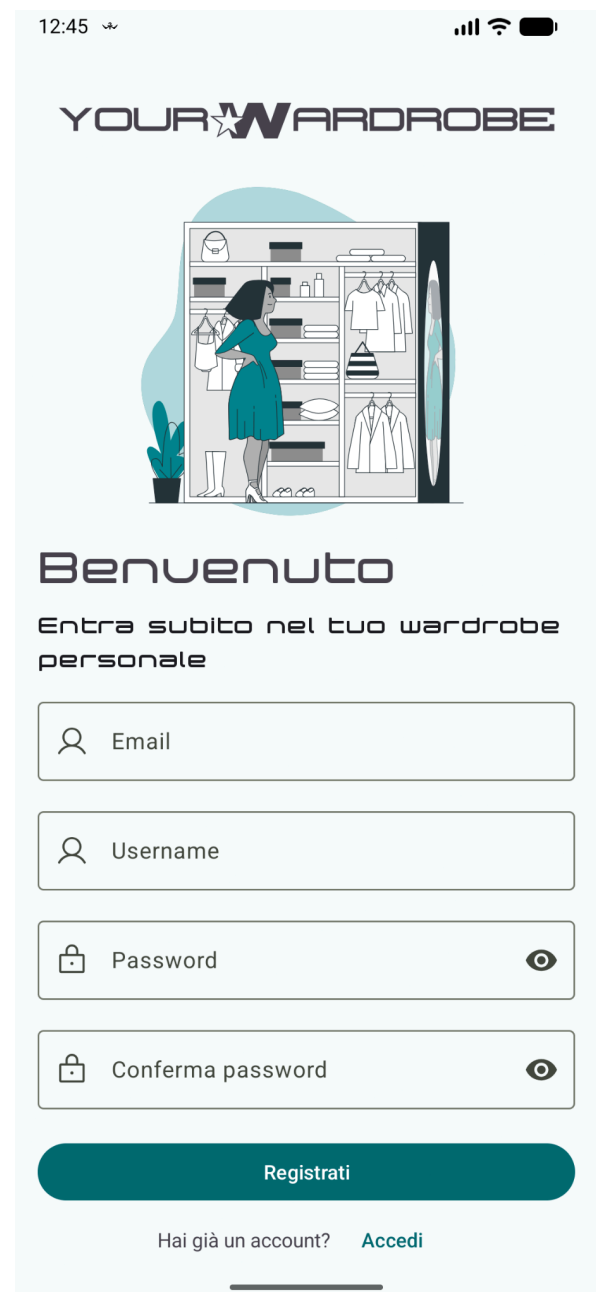
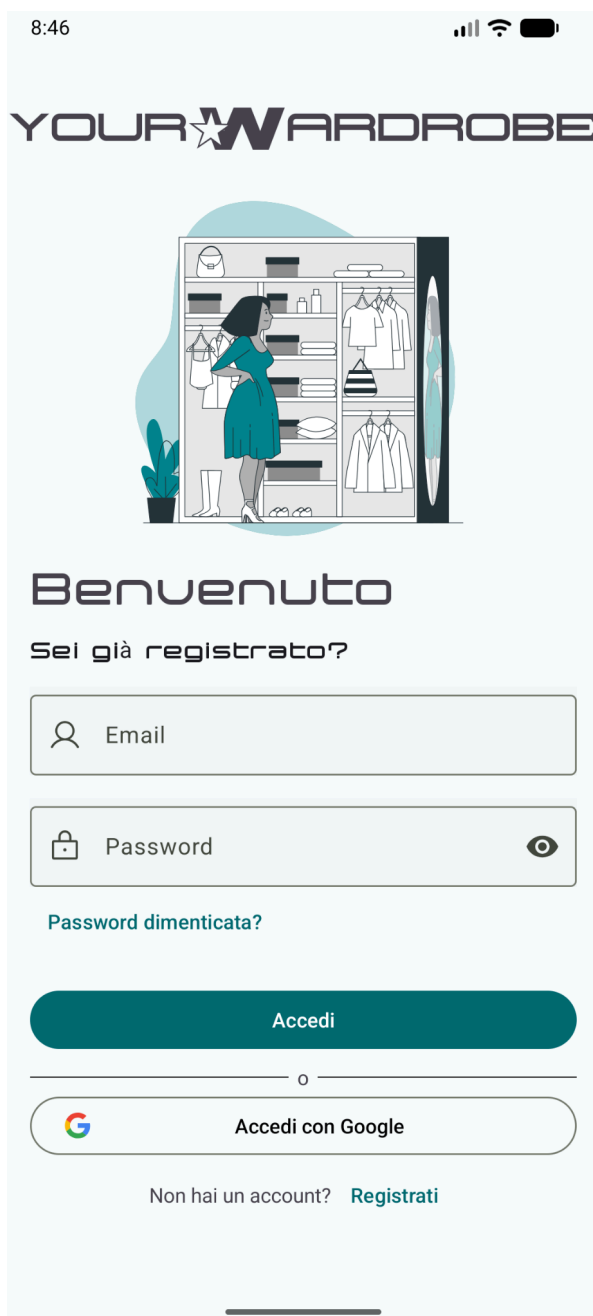
Abbiamo inoltre utilizzato i servizi offerti da **Google ML Kit** per riconoscere se l'immagine fotografata/inserita dall'utente rappresentasse effettivamente un capo e per suggerire la categoria di appartenenza.

Infine, come già citato precedentemente, è stata integrata un API per controllare il meteo del giorno corrente e suggerire outfit adeguati alla temperatura. La nostra scelta è ricaduta su **OpenWeatherAPI**.

Funzionalità

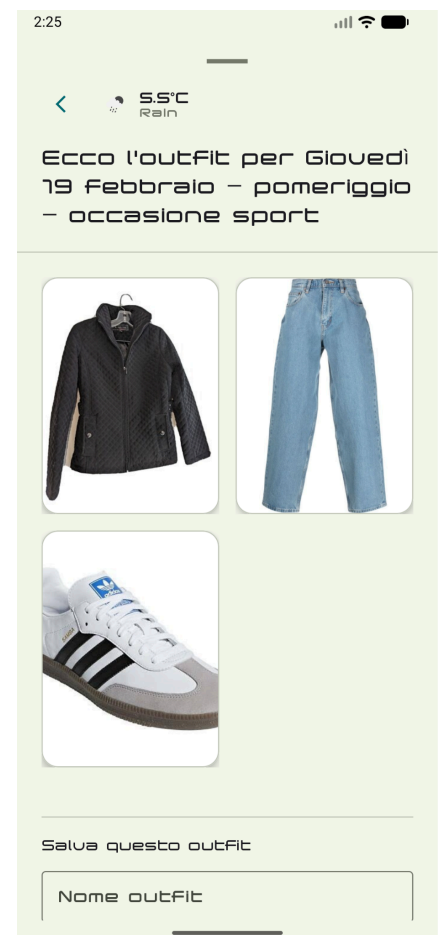
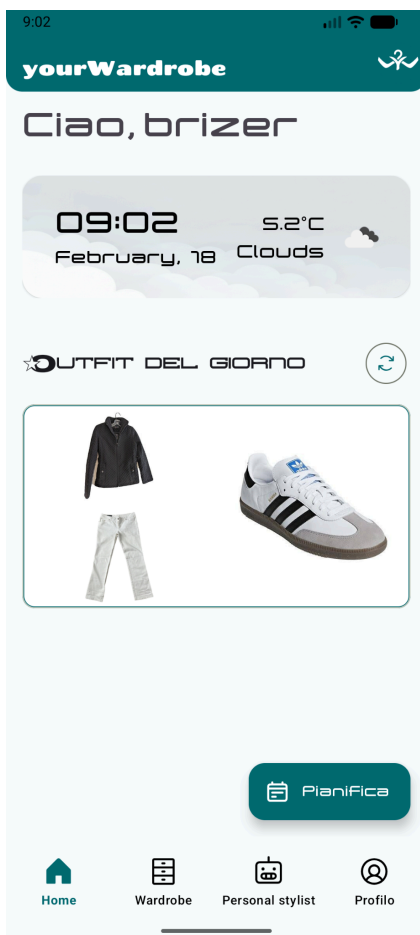
Autenticazione

L'accesso a yourWardrobe è semplice e intuitivo per l'utente. Una volta loggato o registrato, l'utente verrà reindirizzato alla schermata di home page. Per aumentare la facilità di utilizzo e la comodità, se l'autenticazione è stata effettuata in passato sullo stesso dispositivo, l'accesso verrà fatto automaticamente, mostrando la schermata di home. Abbiamo sfruttato le funzionalità offerte da **Google Firebase Authentication**, provvedendo per i nostri utenti un accesso con email+password e con la funzionalità "Accedi con Google".



Home Fragment

L'home fragment è la prima schermata visualizzata una volta aperta l'applicazione. Rappresenta un contenitore di informazioni e feature: l'utente può visualizzare, attraverso una carta dedicata, il meteo quotidiano e ricevere un outfit creato ad hoc per la temperatura esterna. Inoltre, vi è un **Floating Action Button** dedicato per la pianificazione di un outfit per una certa fascia oraria dei prossimi 5 giorni (limite della versione free di **OpenWeather**).

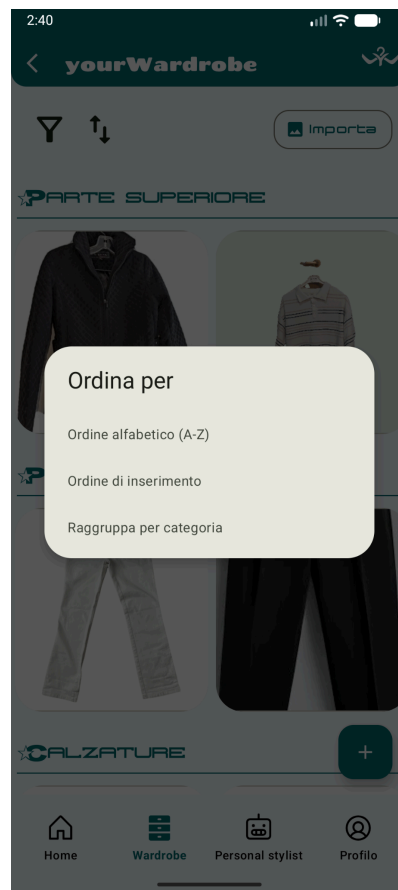


Garment Fragments

Con Garment Fragments vengono intese tutte i fragment che riguardano l'inserimento, la modifica, l'eliminazione e la visualizzazione dei vestiti inseriti dall'utente. Tali funzionalità sono distribuite fra i seguenti fragment:

Clothes Fragment

Clothes Fragment è il fragment dedicato alla visualizzazione dei vestiti dell'utente. I vestiti si presentano raggruppati per macro categorie (**parte superiore, parte inferiore, calzature e accessori**), sono previste funzionalità di ordinamento per ordine alfabetico (nome) e di inserimento. Inoltre, i vestiti sono filtrabili per stile, colore e tessuto. Vi è infine il collegamento ai fragment per l'inserimento di nuovi vestiti da parte dell'utente: Add Garment Fragment e Bulk Import Fragment



Add Garment Fragment

Add Garment Fragment rappresenta il modo più diretto e semplice per l'utente di inserire un nuovo capo nel suo armadio personale. Può essere scattata una foto sul momento oppure inserire una immagine a partire dalla galleria. In entrambi i casi, un label classifier di **Google ML Kit** analizza la foto e riconosce se l'immagine rappresenti un capo o meno. In caso negativo, viene richiesta una conferma all'utente sulla volontà di utilizzare quella specifica immagine.

Dopo aver impostato la foto, è necessario impostare nome, stagione, categoria e sottocategoria, stile e colore. Tutti questi parametri sono fondamentali nella generazione di outfit personalizzati.

9:11

< yourWardrobe

📷

CARATTERISTICHE PRINCIPALI

Nome

Categoria

Categoria

Sottocategoria

+

Stagione

+

Colore

+

Aggiungi capo

2:38

< yourWardrobe

📷

CARATTERISTICHE PRINCIPALI

Nome

felpa

Categoria

Categoria

Parte superiore

Sottocategoria

Felpa X

Stagione

Autunno X

Colore

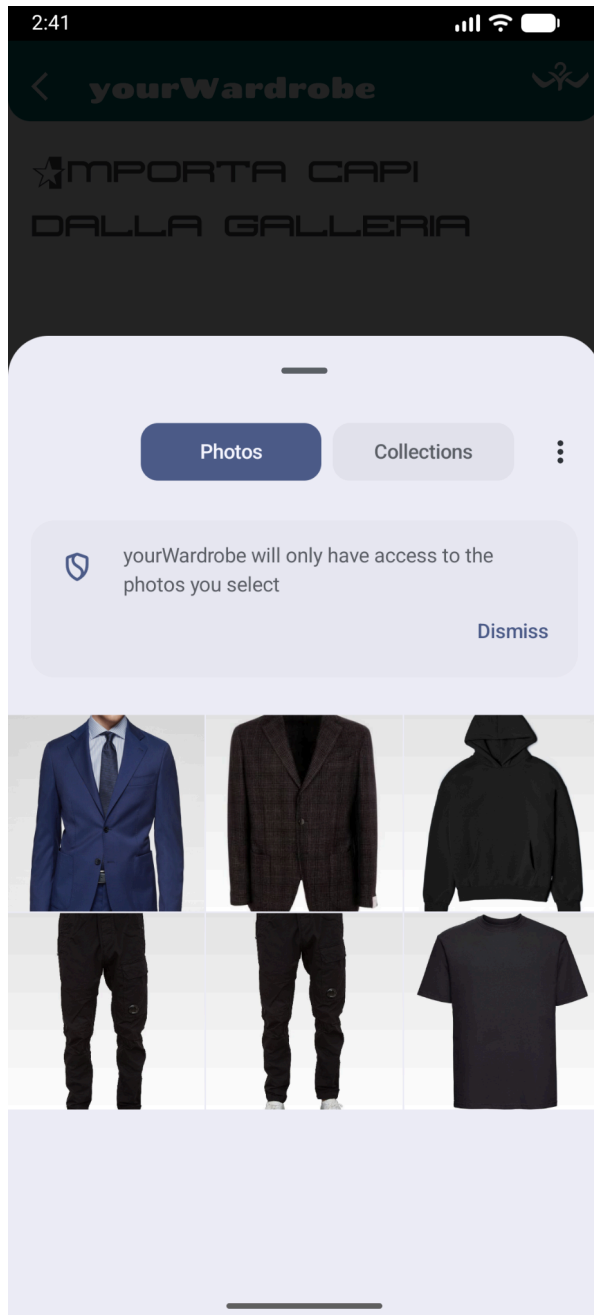
+

Nero X

Aggiungi capo

Bulk Import Fragment

Abbiamo pensato in una fase di sviluppo avanzata che forse, per una grande mole di vestiti da inserire, fosse poco pratico doverne aggiungere uno ad uno manualmente. Per questo, abbiamo implementato una feature adeguata che permette di importare direttamente dalla galleria tutti i vestiti desiderati, per poi salvarli nell'armadio con un approccio di salvataggio più semplice. Google ML Kit è stato sfruttato per suggerire la macro categoria del vestito da aggiungere.



Garment Fragment

La visualizzazione del singolo vestito e delle sue caratteristiche è definita in Garment Fragment. All'interno del fragment è possibile modificare e salvare le caratteristiche (tra cui anche la foto). Naturalmente è possibile anche eliminare il vestito. La sfida più importante nella gestione dell'eliminazione/aggiornamento del capo è stata l'aggiornamento degli outfit contenenti tale capo. Tale problema è stato abilmente risolto attraverso uno script in node.js in costante esecuzione nel firebase, provvedendo all'eliminazione/aggiornamento dell'outfit contenente il capo in questione



Outfit Fragments

Presentiamo ora tutti i fragment per la creazione, visualizzazione, modifica ed eliminazione degli outfit

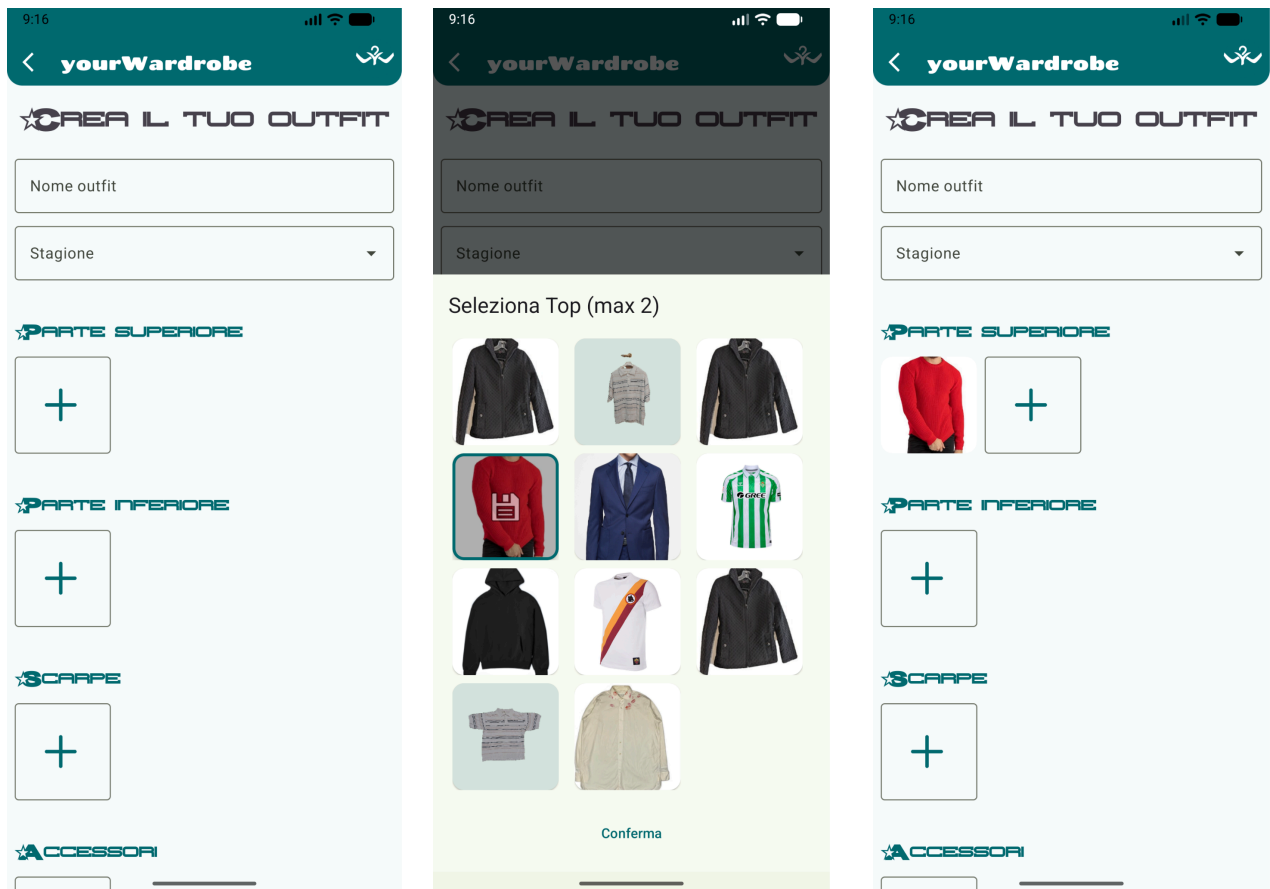
Outfit Menu Fragment

Nel fragment in questione vengono visualizzati tutti gli outfit salvati dall'utente, anch'essi filtrabili e ordinabili. Da questa schermata è possibile, attraverso il FAB dedicato, creare nuovi outfit.



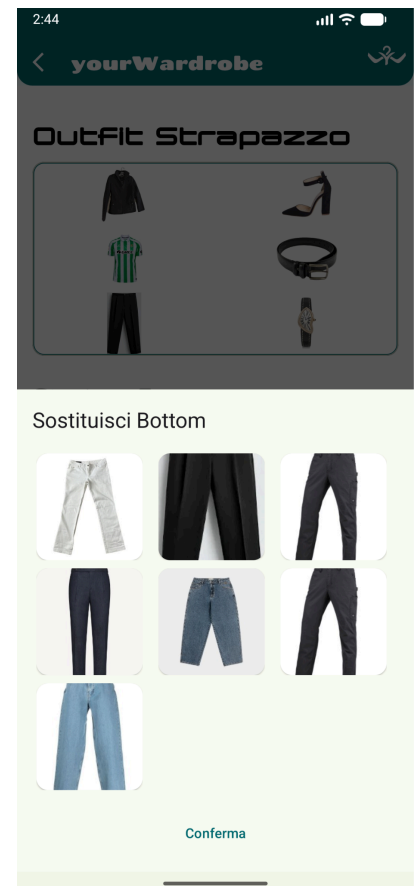
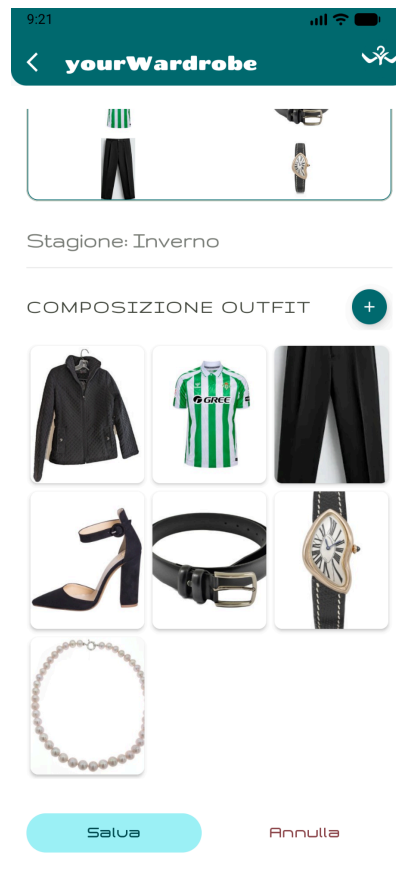
Create Outfit Fragment

In questa schermata è possibile creare semplicemente un nuovo outfit. I capi sono selezionabili attraverso un comodo **BottomSheetDialogFragment**, che mostra i capi divisi per categoria, facilmente selezionabili. Abbiamo pensato di strutturare l'outfit con (massimo) 2 parti superiori, 1 parte inferiore, 1 paio di scarpe e (massimo) 4 accessori. Una volta composto l'outfit, viene fatto inserire all'utente un nome e una stagione. L'outfit viene visualizzato in una comoda carta, che, tentando di mantenere le proporzioni originali, viene composta dinamicamente.



Single Outfit Fragment

La logica di modifica e eliminazione del singolo outfit è presentata nel suddetto fragment. Da qui è possibile aggiungere nuovi componenti nell'outfit e modificare quelli già presenti, attraverso la sostituzione o la rimozione. Vengono naturalmente visualizzati tutti i componenti dell'outfit, attraverso delle opportune carte.



Personal Stylist Fragment

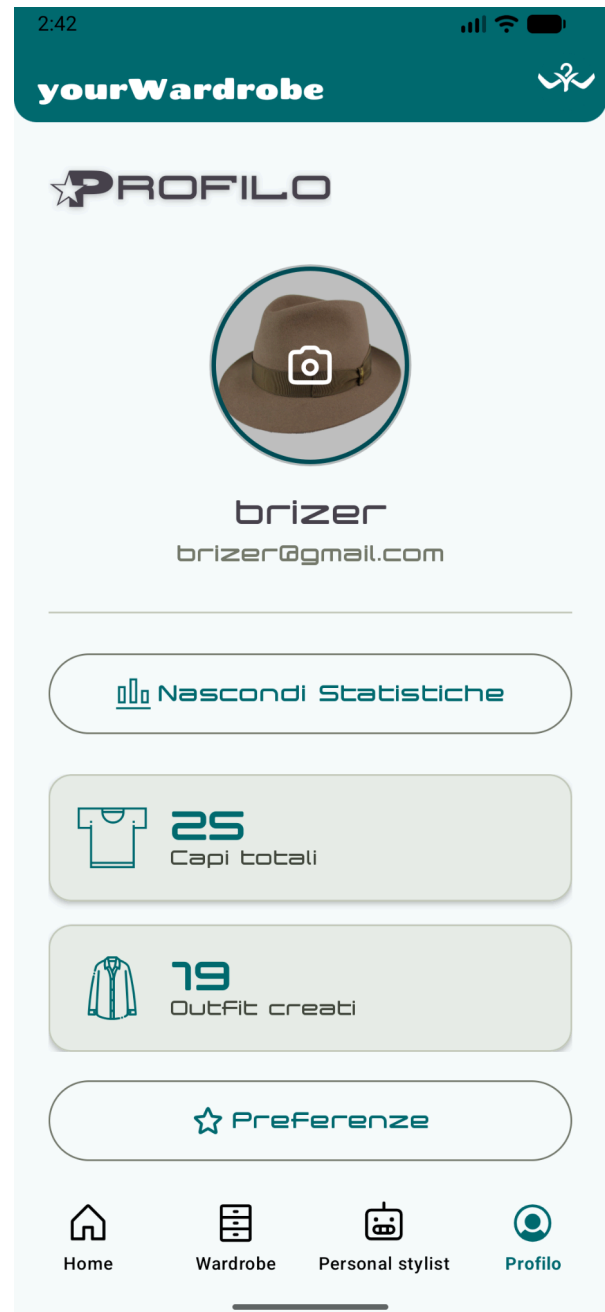
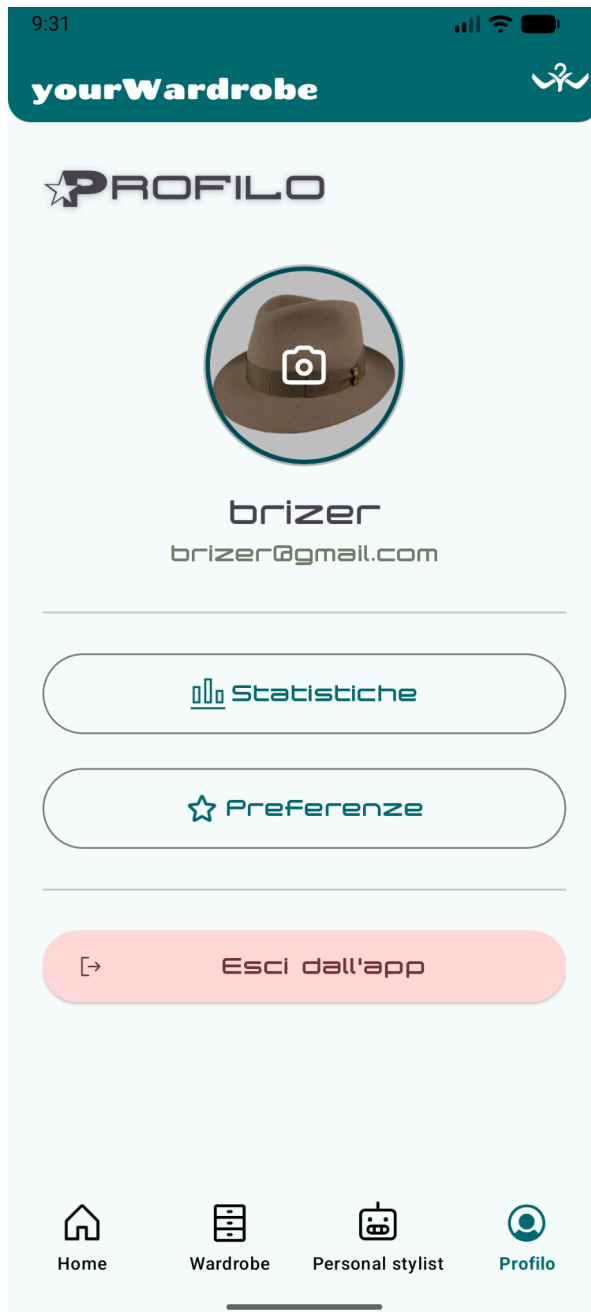
Il Personal Stylist è il cuore "intelligente" dell'applicazione: agisce come un consulente di moda basato sui vestiti dell'utente, suggerendo outfit adatti alla temperatura inserita (di default si considera quella corrente).

L'Algoritmo di Abbinamento provvede a filtrare il guardaroba dell'account per creare combinazioni che rispettino le regole di stile (evitando ad esempio di abbinare troppi colori discordanti o tessuti incompatibili), favorendo le preferenze impostate dall'utente. Viene garantito che l'outfit proposto sia completo, includendo sempre almeno una parte superiore e una inferiore. In caso l'outfit generato fosse di gradimento dell'utente, può essere tranquillamente salvato; in caso contrario, viene offerta la possibilità di rigenerazione.



Profile Fragment

In questa schermata vengono racchiuse tutte le informazioni riguardanti l'utente, quali le sue preferenze, comodamente aggiornabili, statistiche sulla sua esperienza (numero di capi inseriti e outfit inseriti) e la sua foto profilo. è infine presente un tasto per effettuare log out.



Architettura

yourWardrobe segue i principi della **Clean Architecture** attraverso l'implementazione del design pattern **MVVM** (Model-View-ViewModel).

Questa scelta garantisce un elevato disaccoppiamento tra la logica di business e l'interfaccia utente, facilitando la manutenibilità, la testabilità e la scalabilità del software.

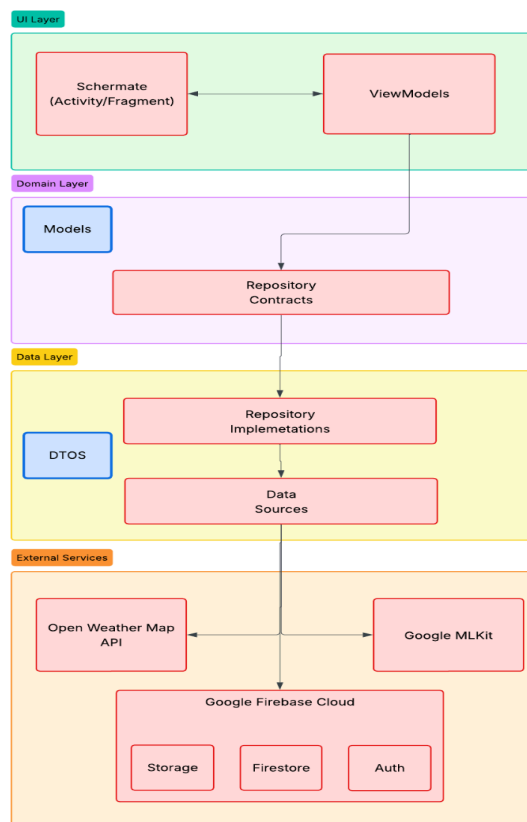
Per la gestione delle dipendenze è stato utilizzato il framework **Dagger Hilt**, che

facilita l'iniezione automatica dei componenti nei vari livelli dell'app.

L'architettura del sistema garantisce la persistenza dei dati tramite la funzionalità **offline first**,

sincronizzando automaticamente le transazioni con il database remoto Firebase non appena la connettività viene ripristinata.

L'applicazione prevede notifiche push all'utente (alle 8:00 del mattino) per suggerire il miglior tipo di outfit in vista al meteo della giornata.



Struttura dei package

model

È tecnicamente compresa in un package più grande, il **domain**, e ha come "fratello" un package **repository** che contiene le interfacce delle repository effettivamente implementate nel package dedicato in **data**. **Model** è composto da classi POJO e rappresenta il cuore pulsante dell'applicazione, contenendo la logica di business:

- Garment: contiene tutte le informazioni riguardanti il singolo capo
- Outfit: contiene una list di capi e tutte le informazioni riguardanti ad un singolo outfit
- User e userPreferences: rappresentano le informazioni riguardanti il generico utente e le sue preferenze
- WeatherInfo: rappresenta le informazioni meteorologiche correnti, ed è utilizzato da personal stylist

Le classi sono state predisposte per favorire comodamente il salvataggio in persistenza su **google firebase**.

adapter

Contiene gli Adapter per le RecyclerView. Il loro compito è prendere una lista di oggetti e "adattarli" graficamente agli elementi della lista definiti negli XML.

data

data è il macro package dove risiede implementazione concreta di come i dati vengono recuperati o salvati.

repository

Il Repository (es. GarmentRepositoryImpl) è il mediatore tra i ViewModel e le sorgenti dati (GarmentRemoteDataSource), esponendo i metodi di accesso ai dati e utilizzando le **Callback** per restituire i risultati in modo asincrono, notificando la UI del successo o del fallimento delle operazioni.

remote

Package che organizza e gestisce tutte le comunicazioni con "l'esterno", interagendo con il firestore per i dati, firebase storage per le immagini e firebase auth per l'utente:

- Auth Remote DataSource: gestisce l'autenticazione tramite Firebase
- Firebase DataSource (es. Garment Remote Data Source, Outfit Remote Data Source): gestiscono l'interazione con il database firestore e firebase storage

dto

I DTO fungono da modelli intermedi per mappare fedelmente i dati grezzi provenienti da API esterne e database, garantendo che le variazioni strutturali specifiche dei servizi cloud siano isolate rispetto alle logiche di business e sui modelli di dominio.

ui

Il package è responsabile della visualizzazione dei dati e dell'interazione con l'utente, organizzato in sottopackage tematici (es. main e welcome).

L'architettura segue rigorosamente il pattern **MVVM**, dove i **ViewModel** gestiscono lo stato della UI in modo reattivo tramite LiveData, garantendo la persistenza dei dati durante il ciclo di vita dei Fragment. L'intero ecosistema è supportato da **Dagger Hilt**, che gestisce l'iniezione delle dipendenze per disaccoppiare la logica di business dai componenti grafici. I **components** (come la CardOutfit) sono View personalizzate e riutilizzabili che incapsulano la logica complessa di disegno e visualizzazione dei collage.

di

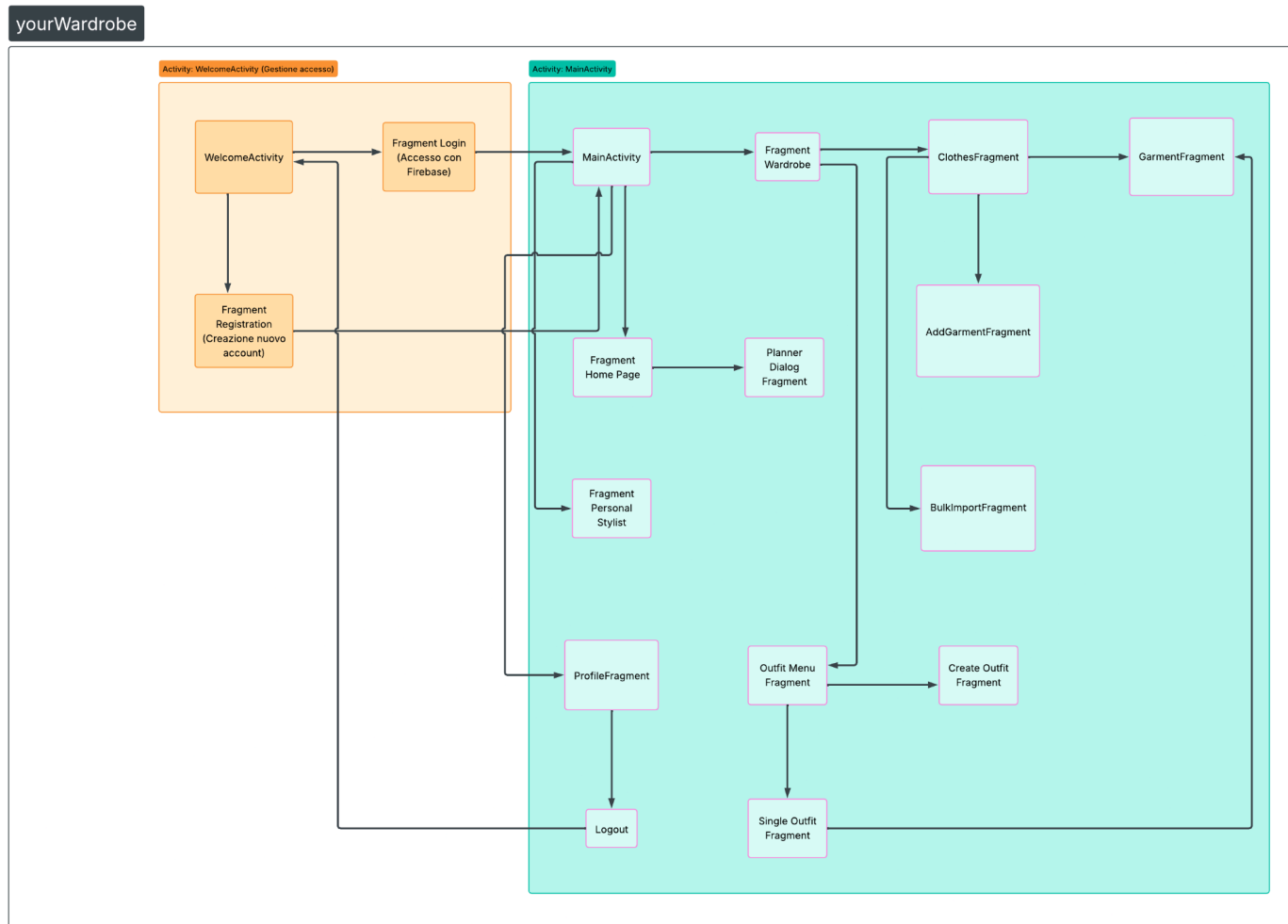
Il package di (dependency injection) racchiude i moduli Dagger Hilt responsabili della configurazione e del provisioning delle dipendenze con scope Singleton.

utils

Il package **utils** contiene classi helper e strumenti trasversali progettati per incapsulare logiche ripetitive e funzionalità tecniche, garantendo un codice più pulito

- **Constants**: Definisce i valori immutabili globali, come gli endpoint delle API OpenWeather.
- **Resource**: Una classe generica utilizzata per incapsulare i dati insieme al loro stato (SUCCESS, ERROR, LOADING), facilitando la comunicazione tra Repository e UI riguardo l'esito delle operazioni asincrone.
- **WeatherUtil**: Fornisce utilità per la manipolazione dei dati meteo.

Design



A sinistra è possibile visualizzare il flow pensato per l'applicazione yourWardrobe: l'utente, una volta registrato o loggato, accede alla main activity che, attraverso una comoda navbar, espone all'utente tutte le funzionalità sopra citate.

Layout

L'applicazione è stata pensata in dya mode e in italiano. Abbiamo utilizzato diversi font per dare all'intero progetto una vibe **y2k/anni 2000**, garantendo una buona leggibilità e conferendo identità all'applicazione. Per i testi principali abbiamo usato un [font](#) custom chiamato "Troglodyte Pop".

★PENULTIMATE

★THE SPIRIT IS WILLING BUT THE FLESH IS WEAK

Per le informazioni testuali secondarie abbiamo utilizzato come coppia di font: [popstar](#) e **chango**.

Testing

Per verificare il corretto funzionamento dell'applicazione in diversi scenari di utilizzo sono stati effettuati test locali automatizzati **junit**.

I test eseguiti sono principalmente **test in isolation** per verificare il comportamento corretto da parte delle classi prese in esame durante le diverse fasi dell'applicazione. Per poter isolare la classe da esaminare, è stato necessario creare delle variabili mock per sostituire le eventuali dipendenze richieste al funzionamento della classe testata, ciò è stato possibile grazie all'uso del framework **Mockito**.

setup classe AddGarmentViewModelTest

```
@Before
public void setUp() {
    when(mockContext.getResources()).thenReturn(mockResources);
    when(mockResources.getStringArray(any(Integer.class))).thenReturn(new String[0]);
    addGarmentViewModel = new AddGarmentViewModel(mockContext, mockGarmentRepository);

    doAnswer(InvocationOnMock invocation -> {
        Callback<Boolean> callback = invocation.getArgument(index: 1);
        callback.onSuccess(data: true); // Simula che l'immagine sia sempre valida
        return null;
    }).when(mockGarmentRepository).validateGarment(any(Bitmap.class), any(Callback.class));
}
```

test di verifica dello stato isEnabled se i campi sono validi in AddGarmentViewModel

```
@Test
public void isEnabled_whenAllFieldsAreValid_isTrue() {

    addGarmentViewModel.isEnabled().observeForever(isButtonEnabledObserver);

    // compila tutti i campi richiesti
    addGarmentViewModel.setGarmentImage(mockBitmap);
    addGarmentViewModel.setGarmentName("Nuova Maglia");
    addGarmentViewModel.setSelectedCategory("Parte superiore");
    addGarmentViewModel.updateSelectedColors(Collections.singletonList("Blu"));
    addGarmentViewModel.updateSelectedStyles(Collections.singletonList("Casual"));
    addGarmentViewModel.setSelectedSeason("Primavera");
    addGarmentViewModel.setSelectedSubCategory("T-shirt");

    verify(isButtonEnabledObserver, org.mockito.Mockito.atLeastOnce()).onChanged(booleanCaptor.capture());
    assertTrue(message: "Il pulsante deve essere abilitato quando tutti i campi sono validi", booleanCaptor.getValue());
}
```


Sviluppi Futuri

Una grande difficoltà incontrata nello sviluppo dell'applicazione è stata la qualità delle immagini fotografate dall'utente e la composizione di un "collage" esteticamente gradevole. La prima feature di uno sviluppo futuro sarebbe sicuramente una opzione di scontornamento della foto inserita dall'utente. Tale feature è presente nel github nel branch *feature/prova*, e comprende tecnologie come **google segmentator e CameraX**.

Un altro possibile sviluppo futuro per **yourWardrobe** potrebbe essere l'implementazione di funzioni social tramite una lista amici per permettere la condivisione di outfit tra utenti, l'aggiunta di una vetrina nella sezione profilo per poter mettere in mostra i propri outfit preferiti e fornire funzionalità di shopping per poter ricercare su una qualche piattaforma di vendita online i capi appartenenti a un outfit in mostra nella vetrina di un amico.

Infine, sarà un fix di alta priorità un miglioramento della versione notturna dell'applicazione e l'integrazione di più lingue.